Global ERP -- Full Flow Technical Documentation

Complete Workshop Flow: Lead to Car Delivery

Version: 2.0 | Updated: 2026-03-31

System Architecture

Tech Stack

Layer	Technology
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS
Backend	Next.js API Routes, PostgreSQL 16
AI	OpenAI GPT (VIN decode, parts suggestion, inspection summary)
Monorepo	Turbo, pnpm
Packages	`ai-core` (business logic), `ui` (components), `ai-tools`
Auth	Session JWT (web), Bearer JWT (mobile)
Real-time	SSE for calls, polling for advisor portal (5-30s)
File Storage	Local filesystem with `/api/files/upload`
PDF	Playwright (Chromium) for HTML-to-PDF rendering

Database: 61+ Tables

Core tables: `leads`, `customers`, `cars`, `inspections`, `estimates`, `job_cards`, `invoices`, `gatepasses`, `purchase_orders`, `inventory_movements`, `part_quotes`, `pis_lead_queue`

Complete Flow (11 Steps)

Step 1: Lead Creation & Booking

API: `POST /api/company/{id}/sales/leads`

Process:

1. Create/find customer (name, phone, email)
2. Create/find car (plate, make, model, year, VIN)
3. Link customer to car (`customer_car_links`)
4. Create lead (type: workshop/rsa/recovery)
5. Set initial stage based on workshop flow:
   - `inspection` flow → stage: `inspection_queue`
   - `direct_estimate` flow → stage: `estimate_pending`

Booking: `POST /api/company/{id}/sales/leads/{leadId}/booking`

1. Create `lead_bookings` record (kind: `workshop_walkin` or `workshop_recovery`)
2. Set scheduled date/time, priority, locations
3. Create pre-inspection form request (`pre_inspection_form_requests`)
4. Send form link to customer (SMS/WhatsApp/Email)
5. Log `lead_booking_saved` event

Pre-Inspection Form: `POST /api/public/pre-inspection/{token}`

1. Customer fills 8 questions (yes/no + details)
2. Accepts terms, provides digital signature
3. Form status: `pending` → `submitted`
4. AI summary generated and synced to lead

Queue Check-In: `POST /api/mobile/company/{id}/queue-system/check-in`

1. Take 4 photos (front, rear, right, left)
2. Take cluster image + 360 video
3. Lead status → `car_in`, stage → `checkin`
4. Store media in `leads.workflow_required` (JSON array)
5. Set `checkin_at` timestamp

Tables affected: `leads`, `customers`, `cars`, `customer_car_links`, `lead_events`, `lead_bookings`, `pre_inspection_form_requests`

Step 2: Advisor Auto-Assignment (PIS Engine)

Trigger: `updateLeadPartial()` detects `lead_status = car_in`

File: `packages/ai-core/src/pis/leadDistribution/autoAssign.ts`

Process:

1. Guard: skip if already assigned or in queue
2. Calculate pipeline value from linked estimate
3. Call `routeLead()` from PIS engine
4. Fetch advisors sorted by composite score DESC
5. Filter out locked advisors
6. Offer to highest-scored available advisor
7. Create `pis_lead_queue` entry (status: `offered`)
8. Set `locked_until` = now + accept window (tier-based)
9. Log `advisor_auto_assigned` lead event

Acceptance: `POST /api/company/{id}/pis/lead-distribution/accept`

1. Update queue: status → `locked`, `accepted_at` = now
2. Set `leads.agent_employee_id` and `assigned_user_id`
3. Log `advisor_accepted` event

Cascade on Timeout:

1. Apply 5-point penalty to timed-out advisor
2. Offer to next best available advisor
3. After 5 cascades → escalate to manual queue

Scoring Algorithm (7 weighted metrics):

Metric	Weight
Conversion Rate	25%
Gross Profit %	20%
Revenue	15%
SLA Compliance	15%
Customer Satisfaction	10%
Call Quality	10%
FOC Penalty	-5%

Tables: `pis_lead_queue`, `pis_lead_queue_history`, `pis_advisor_scores`, `leads`

Step 3: Inspection

API: `GET/POST /api/company/{id}/workshop/inspections`

Process:

1. Inspection created (linked to lead, car, customer)
2. Collect Car stage: verify check-in media, approve/reject
3. Process checks: Oil, Battery, Tyre, OBD (each with status + photos)
4. Vehicle data: VIN decode, plate, make, model, year, tyre sizes, mileage
5. Findings: add parts/services from category catalog
6. Each finding: action (Replace/Service/Repair), priority (Safety Risk/Mandatory/Recommended), qty, photo evidence
7. AI assessment auto-generated per finding
8. Save all → line items created in `line_items` table
9. Complete inspection → generate PDF report

Validation (Step 4 - Vehicle Data): Make, model, year, VIN, plate, tyre sizes (front/rear), mileage all required

Validation (Step 5 - Inspection): All process checks done with photos, all line items saved, all required media uploaded

PDF Generation: `GET /api/company/{id}/workshop/inspections/{inspId}/print`

- HTML template with health scores, findings, evidence, process checks
- Rendered via Playwright to PDF

Tables: `inspections`, `line_items`, `inspection_media`, `inspection_collect_car_review_logs`

Step 4: Estimate

API: `POST /api/company/{id}/workshop/estimates`

Process:

1. Create estimate from inspection (`inspectionId` required)
2. Estimate items auto-populated from inspection line items
3. Set pricing per item: OE/OEM/Aftermarket/Used sale prices
4. AI market pricing analysis available
5. VAT calculated at 5%
6. Customer approval link generated (48-char hex token, 7-day expiry)

Customer Approval: `POST /api/public/estimate-approval/{token}`

1. Customer selects/deselects items
2. Chooses preferred part type per item
3. Accepts terms + digital signature
4. Estimate status → `approved`

Tables: `estimates`, `estimate_items`

Step 5: Parts Approval & Procurement

Vendor Quote: `POST /api/company/{id}/vendors/{vendorId}/part-quotes`

1. Vendor submits pricing (OEM/OE/Aftermarket/Used)
2. Vendor enters part number + uploads diagram (required before viewing order)
3. Quote status: pending → quoted → approved → ordered

Purchase Order: `POST /api/company/{id}/workshop/procurement`

1. PO created from approved quotes
2. Auto-generated PO number (sequential)
3. PO items with quantities and unit costs

Goods Receipt: `POST /api/company/{id}/workshop/procurement/{poId}/receive`

1. Resolve main warehouse location (`resolveMainWarehouse()`)
2. Update PO item `received_qty` and status
3. Create `inventory_movements` record (GRN with `location_id`)
4. Update `inventory_stock` at warehouse location
5. Update `part_quotes.status` → `Received`

Tables: `part_quotes`, `purchase_orders`, `purchase_order_items`, `inventory_movements`, `inventory_stock`, `inventory_locations`

Step 6: Job Card Execution

API: `POST /api/company/{id}/workshop/job-cards`

Stages:

1. Quote Accepted → Done
2. Collect Car → verify media, approve
3. Pre-Work Check → mileage, car details
4. Parts Receive → at least 1 part received, all photos uploaded
5. Start Job → execute work
6. Evidence Upload → working video, completion evidence
7. Completed → mark done
8. Final Inspection → checklist (test drive, cluster, tyre, computer reset, shields) + 4 car photos
9. Car Wash → 5 media files (front, rear, right, left, video)

Tables: `job_cards`, `line_items`

Step 7: Invoice & Payment

Create Invoice: `POST /api/company/{id}/workshop/invoices`

Process:

1. Create invoice from estimate
2. Add service charges as line items:
   - Inspection Fee (from admin config, editable)
   - Labour Charge (editable)
   - Recovery Pickup Fee (if applicable)
   - Recovery Dropoff Fee (if applicable)
3. Recalculate totals: `total_sale`, `final_amount`, `vat_amount`, `grand_total`

Service Charges Config: `pis_config.service_charges`

```
{ "inspection_fee": 150, "recovery_pickup_fee": 200, "recovery_dropoff_fee": 200,
"labour_charge": 0, "currency": "AED" }
```

Payment: `POST /api/company/{id}/workshop/invoices/{invoiceId}/pay`

1. Validate invoice not already paid
2. If wallet payment: check balance >= grand total
3. Deduct from `customers.wallet_amount`
4. Create `customer_wallet_transactions` debit record
5. Set invoice status → `paid`

Wallet Topup: `POST /api/customers/{id}/wallet/transactions`

1. Create credit transaction (amount, method, date, proof)

2. Increase `customers.wallet_amount`

Tables: `invoices`, `invoice_items`, `customers`, `customer_wallet_transactions`

Step 8: Gatepass & Car Release

Create Gatepass: `POST /api/company/{id}/workshop/gatepass`

1. Create from paid invoice
2. Handover type: `branch` or `dropoff_recovery`
3. Status: `pending`

Release Car: `PATCH /api/company/{id}/workshop/gatepass/{gatepassId}`

1. Set status → `released`, `customerSigned` → true
2. Set `supervisorApprovedAt`, `finalNote`
3. Send `release: true` flag → triggers `releaseGatepassAndCloseLead()`
4. Lead status → `closed_won`, `closed_at` set

Tables: `gatepasses`, `leads`

Key Configurations

PIS Config (pis_config table)

Key	Purpose
`score_weights`	Composite score formula
`tier_boundaries`	Elite/Standard/Low thresholds
`commission_rates`	Base/performance/top percentages
`sla_thresholds`	SLA targets per stage
`lead_distribution`	Accept windows, penalties, cascades
`revenue_targets`	Monthly targets
`service_charges`	Inspection/labour/recovery fees

Company Config

Setting	Purpose
`main_warehouse_location_id`	Default receiving warehouse

Security Model

Layer	Mechanism

Layer	Mechanism
Middleware	Session cookie check for all `/api/` routes (except public)
API Routes	`requirePermission()` with RBAC scope (global/company/branch)
Data Isolation	`company_id` filter on all queries
File Access	Auth required for `/api/files/{id}`
Vendor Portal	Separate vendor role with limited permissions
Webhooks	Provider-specific signature verification

Performance Optimizations

Area	Optimization
DB Pool	`max: 15` connections (was 1)
Leads API	Parallel queries via `Promise.all`, pagination (LIMIT 200)
Advisor Portal	Auto-refresh 5s (offers) / 30s (idle)
Inspection	Draft auto-save every 900ms
Queries	Explicit column selection (no SELECT *)
Indexes	`accounting_journal_lines(account_id)`, `leads(customer_id)`

Global ERP System | Version 2.0 | 2026-03-31